SYNCRA

Arquitetura e Decisões Técnicas

Plataforma de criação, distribuição e marketplace de chatbots inteligentes

Campo	Conteúdo
Documento	01 — Arquitetura e Decisões Técnicas
Versão	0.1 — minuta para revisão da equipe
Data de emissão	Agosto de 2026
Situação	Em revisão. Não utilizar como base de implementação até aprovação.
Autores	Vinicius Prado Sobral, Tarso Monteiro Alves Passos, João Guilherme Costa Carvalho, Gabriel Henrique Costa Novaes
Escopo coberto	Fase 1, conforme documento 00 — Visão e Escopo do MVP v1.2
Documentos relacionados	00 — Visão e Escopo · 02 — Contrato de Dados · 03 — Plano de Execução

> **Finalidade deste documento**
>
> Descrever como as peças do sistema se conectam e registrar por que cada escolha técnica foi feita. Não substitui o Contrato de Dados: os esquemas de banco e o formato canônico do bot são definidos no documento 02 e apenas referenciados aqui.

1. Visão Geral

O Syncra é uma aplicação web com backend próprio, dividida em cinco blocos: o estúdio de criação, onde o cliente monta o bot; o repositório de bots, onde a definição é persistida e versionada; o motor de execução, que interpreta essa definição em tempo de conversa; o catálogo, que distribui bots prontos; e a camada de conta, que controla assinatura e consumo.

A separação central da arquitetura é entre definição e execução. O estúdio produz um artefato declarativo — a definição do bot. O motor de execução consome esse artefato. Nenhum dos dois conhece a implementação do outro, e é essa fronteira que permite ter três modos de criação diferentes alimentando um único mecanismo de conversa.

1.1 Camadas

Cliente	Estúdio de criação (React + React Flow) · Painel de gestão · Interface de conversa · Catálogo
▼	
API	Express — autenticação, CRUD de bots, catálogo, assinatura · WebSocket para a conversa
▼	
Domínio	Validação da definição do bot (Zod) · Motor de execução do fluxo · Gestor de contexto · Controle de consumo
▼	

Infra	PostgreSQL (definições, versões, conversas) · Redis (contexto ativo e contadores) · BullMQ (tarefas assíncronas)
▼	
Externo	API Anthropic (inferência) · Provedor de pagamento em sandbox · Canal de mensagens · Sentry

2. Stack por Setor

Setor	Stack	Situação na Fase 1
Backend e orquestração	Node.js, Express, Redis, BullMQ	Ativo
Motor de IA e estúdio	API Anthropic (Claude), Zod, Socket.io	Ativo
Frontend e editor visual	React, React Flow, Zustand	Ativo — prioridade 1
Catálogo e pagamentos	PostgreSQL (Neon), provedor de pagamento em sandbox	Ativo sem transação real
Autenticação	Auth.js ou Clerk	Decisão pendente (ver 8.1)
Infraestrutura	Railway, Vercel, Sentry	Ativo
Widget embarcável	JavaScript puro ou Preact, WebSocket	Fora da Fase 1
Voz	ElevenLabs	Fora da Fase 1

3. Componentes e Responsabilidades

Componente	Responsabilidade	Não é responsável por
Estúdio visual	Editar o grafo de nós e conexões e gravar a definição do bot.	Executar o fluxo ou chamar o modelo.
Gerador por prompt	Traduzir descrição em linguagem natural para a definição do bot, validada antes de persistir.	Persistir definição inválida ou em formato próprio.
IA consultora	Analisar a definição e sugerir melhorias.	Alterar o bot sem confirmação do usuário.
Validador	Garantir que toda definição gravada obedece ao esquema, venha de qual modo vier.	Corrigir silenciosamente definições malformadas.
Motor de execução	Percorrer o grafo, decidir o próximo nó e montar a chamada ao modelo.	Conhecer como o bot foi criado.
Gestor de contexto	Montar a janela de contexto enviada ao modelo a cada turno.	Armazenar histórico permanente — isso é do Postgres.
Controle de consumo	Contar mensagens, aplicar teto e bloquear.	Cobrar ou processar pagamento.
Serviço de assinatura	Receber webhooks e manter o estado ativo ou inativo da conta.	Decidir preço ou emitir cobrança real na Fase 1.
Catálogo	Listar bots da Syncra e copiar a definição para a conta do cliente na aquisição.	Manter vínculo de edição com o bot de origem.

4. Fluxos Principais

4.1 Criação de bot

1. O usuário monta o fluxo no editor visual ou descreve o bot em linguagem natural.
2. No modo prompt, a API Anthropic é chamada com instrução de retornar exclusivamente a definição estruturada.
3. A definição passa pelo validador antes de qualquer gravação. Falha na validação não persiste e retorna erro ao usuário.
4. A definição válida é gravada como nova versão do bot, preservando a anterior.

Consequência prática: como o validador é o único caminho de escrita, um bot criado por prompt é indistinguível de um bot montado à mão no momento em que o editor o abre.

4.2 Execução de uma conversa

5. A mensagem chega pelo canal e é associada a uma conversa existente ou nova.
6. O motor carrega a versão da definição vinculada àquela conversa — não a versão mais recente do bot.
7. O gestor de contexto monta a janela: instrução do nó atual, estado da conversa e trecho recente do histórico.
8. O controle de consumo verifica o teto antes da chamada. Estando bloqueado, não há chamada ao modelo.
9. A resposta é devolvida ao usuário, persistida e o estado da conversa é atualizado.

> **Por que a conversa fica presa a uma versão**
>
> Se o cliente editar o bot no meio de uma conversa em andamento, trocar a definição no ar quebraria o estado do usuário final. Cada conversa carrega a versão com que começou e migra para a nova apenas ao iniciar uma conversa seguinte.

4.3 Aquisição no catálogo

10. O cliente escolhe um bot do catálogo Syncra.
11. O sistema copia a definição para a conta do cliente como um bot novo e independente.
12. A cópia registra o bot de origem apenas como referência informativa, sem herança de alterações.

4.4 Assinatura em ambiente de teste

13. O checkout é iniciado com as credenciais de sandbox do provedor.
14. O provedor confirma por webhook; o endpoint valida a assinatura da requisição antes de aceitar.
15. O estado da conta passa a ativo e os bots vinculados entram em operação.
16. No cancelamento, o mesmo caminho desativa os bots da conta.

Nenhuma condicional de ambiente é escrita no código da aplicação. Sandbox e produção diferem apenas por variáveis de ambiente, o que torna a migração uma troca de configuração.

5. Gestão de Contexto

Manter contexto em múltiplas conversas simultâneas foi identificado como a principal dificuldade do backend, e por isso é tratado como decisão de arquitetura e não como detalhe de implementação.

Camada	O que guarda	Por quê
Redis	Estado da conversa ativa: nó atual, variáveis coletadas e resumo recente.	Leitura a cada turno; precisa ser rápida e descartável.
PostgreSQL	Histórico completo de mensagens e conversas.	Fonte de verdade, auditoria e recuperação após reinício.

A janela enviada ao modelo é montada, não acumulada: cresce até um limite e passa a resumir os turnos antigos. Acumular o histórico inteiro faria o custo por mensagem crescer dentro da mesma conversa, o que é o modo mais silencioso de estourar o orçamento.

6. Custo e Latência

Custo e latência são requisitos de arquitetura nesta fase, não otimizações posteriores. As medidas adotadas:

- Seleção de modelo por tarefa: tarefas simples de roteamento e classificação usam o modelo mais barato; geração de resposta ao usuário final usa o modelo de maior qualidade
- Resposta em streaming na interface de conversa, reduzindo a latência percebida sem alterar a latência real
- Cache de respostas para perguntas de alta repetição, avaliado após a medição do piloto
- Registro de tokens, custo e tempo por chamada desde a primeira versão, separando custo de inferência de custo de canal
- Verificação de teto antes da chamada ao modelo, e não depois

7. Ambientes, Segurança e Conformidade

7.1 Ambientes

Três ambientes: desenvolvimento local, homologação e produção. Toda credencial vive em variável de ambiente. Nenhuma chave de API do modelo trafega para o cliente — as chamadas partem exclusivamente do backend.

7.2 Segurança

- Isolamento por conta em toda consulta: nenhum endpoint retorna bot ou conversa sem filtro por proprietário
- Validação de assinatura em todo webhook recebido
- Limite de requisições por conta nos endpoints de criação e de conversa
- Definição de bot tratada como dado não confiável na execução: o motor interpreta o grafo, nunca executa código arbitrário nele contido

7.3 LGPD

Conversas contêm dados pessoais fornecidos por usuários finais que não são clientes da Syncra. A arquitetura precisa prever, desde já, retenção com prazo definido, exclusão de conversas a pedido e registro do consentimento no início da conversa. Detalhamento fica no documento de conformidade, mas os campos que suportam isso precisam existir no esquema desde a primeira migração.

8. Decisões em Aberto

Os itens abaixo bloqueiam parte da implementação e precisam de decisão da equipe antes da aprovação desta minuta.

#	Decisão pendente	Opções e implicação
8.1	Provedor de autenticação	Clerk entrega mais rápido e cobra por usuário ativo; Auth.js é gratuito e dá controle total, ao custo de mais trabalho inicial.
8.2	Canal de conversa da Fase 1	Web próprio, WhatsApp via Meta, ou ambos. A menção ao custo por conversa da Meta sugere WhatsApp, mas o escopo aprovado prevê interface web. Os dois canais têm modelos de sessão distintos e isso afeta o motor de execução.
8.3	Provedor de pagamento	Mercado Pago consta na análise inicial. Confirmar antes de implementar F-08, já que o formato dos webhooks difere entre provedores.
8.4	Execução de nós no motor	Interpretação direta do grafo a cada turno, ou compilação da definição em uma máquina de estados. A segunda é mais rápida e mais difícil de depurar.
8.5	Estratégia de resumo de contexto	Resumo por modelo, que custa uma chamada extra, ou janela deslizante simples, que perde informação antiga.

9. Controle de Versão

Versão	Data	Descrição	Responsável
0.1	Ago/2026	Minuta inicial para revisão da equipe	Equipe Syncra

Esta minuta só passa a valer como referência de implementação após a resolução das decisões listadas na Seção 8 e aprovação dos quatro integrantes.